V Visweswaran

CSE - III RD year.

Computer Networks

Observation.

436

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
1.		STUDY OF NETWORK COMMANDS	8	
2.		STUDY OF DIFFERENT TYPES OF NETWORK CABLES	8	
3.		EXPERIMENT ON CISCO PACKET TRACER.	9	
4.		SETTING UP OF LAN	9	
5.		EXPERIMENTS ON PACKET CAPTURE TOOL: WIRESHARK	9	
6.		ERROR CORRECTION AT DATA LINK LAYER	9	
7		SLIDING WINDOW PROTOCOL	10	
8		DISCOVER LIVE HOSTS USING NMAP	10	
9.		SUBNETTING IN CISCO PACKET TRACER	10	
10.		INTERNAL WORKING ROUTES IN CISCO		
a)		PACKET TRACER	10	
b)		INTERNET WORKING USING WIRELESS ROUTER, DHCP, INTERNET CLOUD.		
11)a)		SIMULATE STATIC CONFIGURATION USING CISCO PACKET TRACER	10	
b)		SIMULATE RIP USING CISCO PACKET TRACER		
12)a)		END - END COMMUNICATION AT TRANSPORT LAYER	10	
b)		Implement Chat client server session using TCP/IP packets		
13)		Implement your own PING program	9	
14)		Packet Sniffing using RAW Socket	9	
15)		Analyze server using Webalizer		

EXP NO: 01

STUDY OF NETWORK COMMANDS

Windows Command :

1. arp -a : ARP is a short form of Address Resolution Protocol.

OUTPUT:

Interface : 10.16.56.50 .... 0x9

Internet Address	Physical Address	Type
10.16.56.155	ec-12-17-17-12-5c	Dyanamic
10.16.56.295	ff-ff-ff-ff-ff	Static.

2. hostname : This is the simplest of all TCP/IP command. It will display name of computer

OUTPUT: DESKTOP-HSKRCVC

3. ipconfig /all : This command display configuration information about TCP/IP connection.

OUTPUT:

Windows IP Configuration

Host Name ..... : DESKTOP-HSKRCVC

Node Type ..... : Mixed

4. nbtstat -a : This command helps solve the problem with NETBIOS name resolution.

OUTPUT:

Ethernet 2 :

Node IP address : [192.168.66.1] Scope Id : []

Host Not Found.

5. netstat : netstat displays a variety of statistics about computer active TCP/IP connection.

OUTPUT:

Proto	Local Address	Foreign Address	State
TCP	10.16.56.50:49444	4.213.25.242:https	Enabled

6. nslookup : It is used to display DNS details such as IP address of particular computer

OUTPUT:

Server : Unknown
Address : 10.16.56.155

7. Pathping : It is a unique window and its basically a combination of ping and tracert cmd.

OUTPUT : Cmd : pathping -g google.com
Tracing route to google.com [142.250.66.14]
over a max of 30 hops.

8. ping : It is used to check whether one device reach another.

OUTPUT : ping google.com

Pinging

9. Route : It is used to show/manipulate routing table. It is primarily used to setup static routes to specific host or network via host.

Manipulate network Routing Table.

Route : [-f] [-p] [-4] [-6] command

13.:. 00 00 27 00 00 00 ..

⇒ LINUX COMMANDS:

1. IP commands

i) cmd : ip address show - display all IP address of network interface

OUTPUT:

etho : inet 192.168:1.10/24

ii) ip address add 192.168.1.254/14 - Assign an IP address to an interface

OUTPUT: IP address added successfully.

iii) ip link set etho up - Enables the network interface

OUTPUT: etho state up.

iv) ip link set etho down - Disables the network interface

OUTPUT: etho state down

v) ip link set etho promise on - Enables Promiscuous mode

OUTPUT: Promiscuous mode enabled

vi) IP route add default via 192.168.1.254 dev etho - Adds the default gateway.

vii) IP route get 10.10.1.4 - shows the route to a destination IP.

OUTPUT: 10.10.1.4 via 192.168.1.254 dev etho

2. ifconfig: Displays network interface configuration.

OUTPUT: etho link encap: Ethernet inet 192.168.1.0

3. mtr

i) mtr google.com : Shows route statistics to host

OUTPUT: HOST: localhost loss %. avg Best worst.

ii) mtr -g google.com - Shows numeric IP address only

OUTPUT : 142.250.183.286

iii) mtr -b google.com - Shows both hostname & IP address

OUTPUT : google.com [142.250.183.206]

iv) mtr -c 10 google.com - Sends only 10 packet

OUTPUT : google.com [142.250.183.206]

4. tcpdump

i) dnf install -g tcpdump - Install tcpdump

ii) tcpdump -D - List available capture interface.

OUTPUT : 1. etho 2. IO

iii) tcpdump -i etho -c 10 :- capture packets to/from host

OUTPUT : 10 Packets captured.

iv) tcpdump -i etho src host 88.88 - capture packets from host

OUTPUT : 88.88 > 192.168.1.10

v) tcpdump -i etho dst host 8.8.8.8 - captures packets to a host

OUTPUT : 192.168.1.10 > 8.8.8.8

vi) tcpdump -i etho port 53 - Captures DNS traffic

OUTPUT : DNS captured

vii) tcpdump -i etho -c host www.google.com / and port 443 - capture HTTPS product.

OUTPUT : TCP 443 captured

viii) tcpdump -i eth0 port not 53 and not 25 - Capture all traffic except DNS and SMTP.

OUTPUT: TCP/IP packets displayed.

5. ping.

i) ping google.com - checks connectivity to host

OUTPUT: 64 bytes from google.com time = 12ms.

ii) ping -c 10 google.com - sends only 10 ping request

OUTPUT: 10 packets transmitted, 10 received.

6. nmcli

i) nmcli connection show - lists all network connections

Output: wired connection 1.

ii) nmcli connection modify "wired connection" - modifies existing connection.

iii) nmcli connection show - Display connection details

Output: IPU. method auto

iv) nmcli connection modify "wired connection 1" IPU4 method auto - configure DHCP

Output: IPU4 method set to auto

v) nmcli connection modify "wired connection 1" IPU4. method manual IPU4 addresses 192.02.1/24 IPU4. gateway 192.0.2.254 IPU4. DNS 192.0.2.206 - configures static IPU4 address.

OUTPUT: IPU4 settings updated.

vi) nmcli connection modify "wired connection" IPV6 method auto - Configure automatic IPV6

Output : IPV6 method set to auto

vii) nmcli connection up Internal - LAN : Activate the connection profile.

EX NO: 02

* AIM :

* TYPES

Cable

Type

UTP

STP

SSTP

* RESULT :

The above windows commands and LINUX Commands are executed and outputs are completed and verified successfully

Ex No: 02

STUDY OF DIFFERENT TYPES OF NETWORK CABELS.

* AIM:

To understand different types of network cables.

* TYPES:

Different types of cables used in networking

1. Unshielded Twisted Pair [UTP] cable
2. Shielded Twisted Pair [STP] cable
3. Coaxial cable
4. Fibre optic cable

Cable Type	Category	Maximum Data Transmission	Advantages & Disadvantages	Applications
UTP	Category 3	10 bps	Advantages: • Cheaper in cost • Easy to install as they have smaller diameter Disadvantages: • More prone to EMI	10Base-T Ethernet Fast Ethernet, Gigabit Ethernet
	Category 5	Upto 100 Mbps		
	Category 5e	1 Gbps.		
STP	Category 6	10 Gbps	Advantages: • Shielded • Faster than UTP • Less susceptible to noise Disadvantages • Expensive • Greater installation effort	
SSTP	Category 7	10 Gbps		

Coaxial Cable	RG-6 RG-59 RG-11	10 - 100 Mbps	Advantages • High bandwidth • Immune to interference • Low loss bandwidth • Versatile Disadvantages • Limited distance • Cost • Size is bulky	→ Speed of signal → Television network → High speed internet connection
Fibre Optics Cable	Single mode Multi mode	100 Gbps	Advantages • High speed • High bandwidth • High security • Long distance Disadvantages • Expensive • Requires skilled installers	Maximum distance of fibre optics cable is around 100 meters

* RESULT :

Thus, the different types of network cables were studied and understood successfully.

EXP NO : 3

EXPERIMENT ON CISCO PACKET TRACER

* AIM :

To understand its environment and behaviour of network devices of CISCO Packets Tracer Simulator

i) Peer - to - Peer connection : (P2P) :

Cross over cables were used to connect PC and laptop. Only a cross over cable can enable the communication directly between same type of end devices. Usage of such cables have been replaced by switches, hubs, Routers

2. Hub connection :

A hub connects multiple devices in a network by blindly broadcasting all incoming data signals to every connected devices.

3) Switch connection :

Switch connects multiple devices within single LAN file and intelligently towards data packets directly to specific destination using MAC address.

4. Router connection :

A router connects different subnets distinct networks together, directly data traffic between them using IP address and default gateways.

* RESULT :

Hence the environment of CISCO PACKET TRACER and behaviour of all network devices in the software were studied, simulated and verified successfully.

EXP NO : 04

SETTING UP A LOCAL AREA NETWORK.

* AIM :

To setup and configure a LAN connecting four computers through a network, assign static IP addresses to all devices, verify connectivity using ping command and demonstrate resource / data.

* Equipment Required :

S.No	Equipment	Quantity	Specification
1.	Personal Computer	4	Ethernet onboarded
2.	Network Switch	1	8/16/24 port
3.	Ethernet Cables (cat 5e / cat6)	4	Straight - through, RJ - 45 Connectors.

* IP address configuration table :

Device	IP address	Subnet mask	Connected to
PC0	10.1.1.1	255.0.0.0	Switch port Fa 0/1
PC1	10.1.1.2	255.0.0.0	Switch port Fa 0/2
PC2	10.1.1.3	255.0.0.0	Switch port Fa 0/3.
PC3	10.1.1.4	255.0.0.0	Switch port Fa 0/4.
Switch	10.1.1.5	255.0.0.0	VLAN 1 interface

* Implementation in Cisco Packet Tracer:

Step 1: Place the device

1. Open Cisco Packet Tracer
2. From device category panel select end device
3. Select switches and drag 1 switch

Step 2: Cable the devices

4. Click connections → select copper straight
5. Click each PC, then click switch and select available fast ethernet port to complete each connection
6. Repeat all 4 PC's. Green light confirms a successful physical connection.

Step 3: Assign IP address to PC's

7. Click PC → desktop tab → IP configuration
8. Enter IP address and Subnet mask as per IP table

Step 4: Configure the switch management

9. Click the switch → CLI tab.
10. Enter following commands to assign management IP:
    i) enable
    ii) configure terminal
    iii) interface VLAN 1
    iv) IP address 10.1.1.5 255.0.0.0
    v) no shutdown
    vi) exit.

Step 5 : Verify connection using ping

11. Click PC → desktop tab → command prompt
12. Use ping command to test connectivity to other PC's eg : ping 10.1.1.2.
13. Successful "Reply from ..." messages confirm connectivity.

Step 6 : Demonstrate Data sharing

14. Add server device and connect it to switch
15. Assign IP address
16. Open the server's services tab and enable HTTP service
17. From another PC, open the web browser and enter server IP address to load page.

Step 7 : Observe Packet Flow in Simulation mode.

18. Switch to simulation mode.
19. Send a ping request between two devices and use play / capture controls to step through packet as it travels hop-by-hop through the switch observing MAC-address based forwarding.

* Result :

Thus the LAN was configured and verified successfully.

EXPERIMENTS ON PACKET CAPTURE
TOOL : WIRESHARK.

* AIM :

Experiments on packet capture tool : wireshark.

* Wireshark :

It is a network analysis tool formally known as Ethereal, captures packets in real time and display them in human-readable format.

* Procedure :

Capturing Packets :

After downloading and installing, launch it and double click the name of network interface under, capture to start capturing packets on that interface. As soon as we click the interface's name, you see packets start to appear in real time. It captures each packets sent to or from your system. Always make promiscuous mode enabled which is enabled by default, to check click capture → options and verify checkboxes are activated. Create a filter to display only TCP packets, inspect packets & provide flow graph.

* Select local area connection in wireshark
* Go to capture option
* Select stop capture automatically after
* Then click start capture
* Search TCP packets in search box
* To see flow graph, click statistics flow graph

* Save packets.

Likewise, search diff packets and save those

* Result:

Thus the packets are captured and filtered using wireshark.

Exp No: 06

ERROR CONNECTION AT DATA LINK LAYER.

* AIM :

To write a program that implement error detection and correction using Hamming code concept. Make a test run to input data stream and verify error correction feature.

* Sender.py :-

```
def find_r(k):
    r = 0
    while 2**r < k+r+1
        r+=1
    return r
text = input("enter text: ")
binary = ""
for ch in text:
    binary += format(ord(ch), "08b")
print("Binary: ", binary)
k = len(binary)
r = find_r(k)
print("Data bits: ", k)
print("Redundancy bits: ", r)
r = k+r
code = [0]*(n+1)
s = 0
for i in range(1, n+1):
    if i & (i-1) != 0:
        code[i] = int(binary[j])
        j+=1
p = 1
```

```
while p<=h:
    parity = 0
    for i in range (p,h+1):
        if i&p!=0:
            parity^= code[i]
    code[p] = parity
    p = p*2
hamming_code = ""
for i in range (1,h+1):
    hamming_code += str(code[i])
print("hamming code : ", hamming_code)
with open("channel.txt", "w") as f:
    f.write(str[k] + "\n")
    f.write(hamming_code)
print("Data saved in channel.txt")
```

* Receiver.py

```
def binary_to_text(binary):
    text = ""
    for i in range (0, len(binary), 8):
        byte = binary[i:i+8]
        if len(byte) == 8:
            text += chr(int(byte,2))
    return text
with open("channel.txt","r") as f:
    k = int(f.readline())
    data = f.readline().strip()
print("Received Hamming code", data)
h = len(data)
```

```
code = [0] + [int(x) for x in data]
corrupted_binary = ""
for i in range(1, h+1):
    if i*(i-1)!=0:
        corrupted_binary += str(code[i])
corrupted_binary = corrupted_binary[1]
corrupted_text = binary_to_text(corrupted_binary)
print("corrupted text :", corrupted_text)
error = 0
p = 1
while p<=h:
    parity = 0
    for i in range(p, h+1):
        if i&p:
            parity ^= code[i]
    if parity:
        error += p
    p = p*2
if error==0:
    print("No error detected")
else:
    print("error detected at position.", error)
    code[error] = 1- code[error]
    print("error detected")
Corrected_binary = ""
for i in range(1, h+1):
    if i & (i-1)!=0:
        corrected_binary += str[code[i]]
Corrected_binary = corrected_binary[:k]
```

```
corrected_text = binary_to_text(corrected_binary)
print("Corrected text:", corrected_text)
```

* Result:

Thus, error detection and correction is done using Hamming code.

EXP NO: 9

SLIDING WINDOW PROTOCOL

* Aim:

To write a program to implement flow control at data link layer using Sliding window Protocol. Simulate the flow of frames from one node to another

→ Create a sender program with following features:

1) Input window size from the user
2) Input a text message from the user
3) Consider 1 character per frame.
4) Create a frame with following fields
5) Send the frames. [Print output in screen & save it in a file called sender-buffer]
6) Wait for acknowledgement from receiver.
7) Reader a file called receiver buffer
8) Check ACK filed the acknowledgement number
9) If acknowledgement number is as expected.

→ Create a receiver file with following features:

1) Read a file called sender-buffer
2) Check frame number
3) If frame no is as expected, write appropriate ACK no in receiver-buffer else write NACK no in receiver-buffer file.

⇒ Sender code :

```
import os
import time
Base_dir = os.Path.dirname(os.Path.abspath())
print("Sender buffer", sender_file)
Print("Receiver buffer", receiver_file)
window_size = int(input("Enter window size : "))
message = input("Enter text message : ")
if window_size <= 0 :
        print("window size must be greater than 0")
        false systemed ;
frames []
while start < total_frames :
    end = minstart + window_size, total_frames)
    print("\n ======= ")
    print("Sending frames", start+1, "do", end)
    print("\n ======")
elit_ack.type == "NACK"
    Print("NACK received")
    point("Retransmitting from frames : ", ack_no)
else :
    Print("Unknown response : Retrying current window")
print("\n ======== ")
Print("Total frames", total_frames)
```

⇒ Receiver Program :

```
import time
import os
expected = 0
while true:
    if not os.path.exists("sender_buffer.txt"):
        time.sleep(1)
        continue
    with open("send_buffer.txt") as f:
        frames = f.readlines()
    if not frames:
        time.sleep(1)
        continue
    error = false
    for line in frames:
        n, data = line.strip().split(",")
        n = int(n)
        print("Received", n, data)
    if n != expected:
        with open("Receiver_buffer_text", "w") as f:
            f.write("NACK:" + str(expected))
        error = true
        break
    expected += 1
if not error:
    with open("Receiver_buffer_text", "w") as f:
        f.write("ACK" + str(expected))
open("sender_buffer_text", "w").close()
time.sleep(1)
```

Sender code input :

window size : 3

Message : HELLO

Sender code output :

Frame 0 : H

1 : E

2 : L

3 : L

4 : O

Receiver Code Input :

Sender - buffer - test

0, H

1, E

2, L

3, L

4, O

Receiver code Output

Received : 0 H

1 E

2 L

3 L

4 O

9/10

→ Result:

The sliding window protocol was successfully implemented to control frame flow

Exp No: 08

DISCOVER LIVE HOSTS USING NMAP.

* Aim:

To discover live host using Nmap scan (ARP, ICMP, TCPLOOP) on the tryhackme platform.

* Subnetworks:

A network segment is a group of computers linked through a shared medium like an Ethernet switch or wi-fi access point. In IP networks, a subnetwork is a logical connection.

In the provided network diagram, there are four network segments or subnetwork, your system would manually connected to Security policies. The figure displays two Subnets

i) 116 Subnets
ii) 124 Subnets

These features a subnet mask of 255.255.255.0

Keys to Run compiler:

i) from Computer 1
ii) To Computer 1
iii) Packtype = "ARP Request"
iv) Data Computer

* Discovering live host:

TCP/IP layers:

i) ARP from link layer
ii) ICMP from network layer

iii) TCP from Transport Layer

iv) UDP from Transport Layer.

Send a packet with following:

i) from computer 1

ii) To computer 3

NMAP host discovery using ARP:

How we can determine which host are operational? Its crucial to prevent unnecessary port scanning on hosts that are offline. This allows you to discover live system on same subnet as you target machine.

10/10

* Result:

Therefore the discover live host using Nmap scan platform link was executed successfully.

EXP NO: 09

SUBNETTING IN CISCO PACKET TRACER.

* AIM:

Implementation of Subnetting in CISCO PACKET TRACER Simulator.

* Procedure:

i) Classical IP subnetting is a technique that allows for more efficient use of IP address by allowing for subnet masks networks.

ii) Creating a network topology:

The first step in implementing classless IP subnetting is to create a network topology in packet Trace. To create a network technique to add devices.

iii) Adding the devices:

One we have created our network topology, we can add devices to it.

iv) Subnetting:

To subnet the network address of 192.168.1.0/24 to provide enough space for atleast 5 address each.

The IP addressing for network show in the.

i) Router R1

ii) Gigabit Ethernet 0/0 : 192.168.1.1

iii) Gigabit Ethernet 0/1 : 192.168.2.1

iv) Switch S1

vi) PC1 : 192.168.1.11

vii) PC2 : 192.168.1.12

viii) PC3 : 192.168.1.13

ix) PC4 : 192.168.1.14

x) PC5 : 192.168.1.15

xi) Fast Ethernet 0/0 : 192.168.2.0/27

xii) PC5 : 192.168.4.15.

Configure the device :

enable
configure terminal
interface fastEthernet 0/0
ip address {IP address} {subnet mask}
no shutdown
exit
Interface FastEthernet 0/1
IP address {IP address} {subnet mask}
no shutdown
exit

* Result :

Therefore, subnetting simulation was executed successfully.

Exp No: 10

Interral Working Routes in CISCO Packet Tracer.

* Aim :

To design and configure a simple Internals work using a router in cisco packet tracer and fast connectivity between two PC's.

* Procedure :

Step 1: Configure router 1

* Select the router and open CLI
* Press enter to start configure router
* Type enable to activate privelage mode.

Step 2: Assign IP address to every PC in network.

* Select the PC, go to desktop and select IP configure and assign

* IP configure and assign IP address default gateway

* Assigns default gateway of PC's : 192.168.10.1
* Assigns default gateway of Res 192.168.20.1

Step 3 :

* Connect fast ethernet a port of P10 with fast ethernet 0/0 port of router 1 using copper straigh through cable.

* Connect fast ethernet 0 power with fast ethernet all port of row using copper straight through cable.

* Result :

Therefore, internet is working router in Cisco packet tracer.

Internet working using wireless Router, DHCP.

* AIM:

To design and configure an Internet working using a wireless router, DHCP server and internet cloud.

* Procedure:

i) Open CISCO packet tracer and create required network topology

ii) Connect PC: laptop wireless router, cable, modem, internet cloud server.

iii) Configure laptop to wireless router and PC to DHCP.

iv) Configure server with IP 208.87.220.220 and enable DHCP and DNS.

v) Verify connectivity using IP configure and ping CISCO.com.

* Result:

Therefore, the internetwork was successfully designed and configured using wireless router, DHCP server and interpret cloud.

Exp No: 11

Simulate Static Routing Configuration using CISCO packet tracer.

* AIM:

To simulate static routing configuration using Cisco packet tracer.

* Procedure:

* Open CISCO packet tracer and create the given network topology
* Connect 3 routers and PC's using suitable cable.
* Connect IP address for router and PC's
* Enable an router interface using no shutdown
* Configure static routers on router 0 to router 1, router 2 using ip mobiles.
* Set main & backup router using difference administrative distance values
* Test connectivity
* Use tracer verify the routing path
* Disconnect the main link and verify the backup

* Result:

Therefore, the simulate static routing configuration using CISCO packet

Exp No: 11) b)

Simulate RIP using Cisco Packet Tracer

* Aim:

To simulate RIP using 3 routers and 2 Pc's cisco packet tracer.

* Procedure:

i) Create the topology with 3 routers and 2 Pc's
ii) Connect device using suitable cables
iii) Assign given IP address to both Pc's
iv) Configure fast element and serial interface of all routers.
v) Enable interface using no shutdown
vi) Set clock execute of 4000 and bandwidth to on DE Serial interface
vii) Enable RIP on all router using router plan
viii) Add directly connected network using ring Network commands.
ix) Test connectivity using ping 20.0.0.8.
x) Use trace to 20.0.0.2 to verify the selected route
xi) Disconnect the direct creates - 0 - router to link and router again to verify RIP Selait allocated route.

* Result :

Thus the simulation of RIP has been successfully done.

EXP : 12 END TO END COMMUNICATION AT TRANSPORT LAYER

a) Implement ECHO client server using TCP packets

ECHO SERVER :

Code :

```
Import Socket
S = Socket.Socket()
S.bind(("localhost", 5000))
S.listen(1)
Cladder = S.accept()
msg = C.recv(1024).decode()
Print("Client :", msg)
C.send(msg.encode())
S.close()
C.close()
```

ECHO client :

```
Import Socket
S = Socket.Socket()
S.connect(("localhost", 5000))
msg = input("Enter message")
S.send("msg:encode())
Print("Echo : ", S.recv(1024).decode())
S.close()
```

Input :

Enter message : Hello server

Output :

Server

Client connected

Received : Hello server

Echo from server : Hello server

* Result :

The Echo client server using TCP packets was studied and executed successfully.

Exp : 12 b) Implement chat client server using TCP/IP packets.

AIM :

To implement a chat client server application using TCP packets.

CODE :

SERVER

```
Import socket
s = socket.socket()
s.bind (("localhost", 5000))
s.listen()
c, a = s.accept()
while True :
        msg = c.recv(1024).
        print ("Client :", msg)
        if msg == "bye" :
                break
        reply = input ("server")
        c.send (reply.encode())
c.close()
```

CLIENT

```
Import socket
s = socket.socket()
s.connect (("localhost", 5000))
while TRUE :
        msg = input ("
```

```
if msg == "bye":
    break
print("Server: ", s.recv(1024).decode())
s.close()
```

INPUT

Client: Hello
Server: Hi
Client: Bye

OUTPUT

Client: Hello
Server: Hi

Result:

The chat client server applications were successfully studied and implemented using TCP packets.

Exp 13 : Implement your Own Ping Program.

* AIM :

To implement a ping program using python to check the connectivity between source and destination host.

* ALGORITHM :

1. Get the destination IP address
2. Send request to destination
3. Wait for response
4. Calculate response time
5. Display whether host is reachable
6. Repeat process for few request.

* CODE :

```
import os
import time
host = input ("Enter host :")
for i in range (4):
    start = time.time()
    result = os.system ("ping -n 1" + host + ">
                        null")
    end = time.time()
    if result == 0 :
        print ("Reply from", host,
    "Time =", round ((end-start)* 1000, 2),
                "ms")
    else :
        Print ("Request timed out")
time.sleep(1)
```

Input :

Enter host : 127.0.0.1

Output :

Reply from 127.0.0.1 Time = 1.23 ms

Reply from 127.0.0.1 Time = 0.98 ms.

* Result:

The ping program was successfully studied and implemented.

EX NO : 14

PACKET SNIFFING USING RAW SOCKET

* AIM :

To write a program using RAW socket to capture and display network packets.

* ALGORITHM :

1. Create a RAW socket
2. Bind the socket to network interface
3. Receive packets from network
4. Extract IP header info
5. Display source IP, destination IP & protocol
6. Continue capturing packets

* CODE :

```
import socket
S = socket.socket(socket.AF_INET, socket.SOCK_RAW,
                  socket.IPPROTO_IP)
S.bind(("0.0.0.0", 0))
S.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
print("Packet Sniffer started")
while TRUE:
    data, addr = S.recv from (65535)
    print("Packet received from: ", addr[0])
```

INPUT :

Network Packets

OUTPUT :

Packet Sniffer started

Packet received from : 192.168.1.5

Packet received from : 192.168.1.1

8/11/X 9/10

* RESULT :

The RAW SOCKET SNIFFING PROGRAM was Successfully implemented

Exp 15 :

ANALYZE SERVERS USING WEBALIZER

* AIM :

To analyze different types of web logs using Webalizer Tool.

* Procedure :

i) Run Webalizer Windows Version
ii) Input web log file
iii) Press Run Webalizer

* RESULT :

Thus, web logs using webalizer was successfully analyzed.